Measurement-efficient regulation of drifting multi-channel hardware: an open call to reproduce on physical devices

TrueLoop Compute, NEOTECH Inc., Port Coquitlam, BC
Evaluation key: compute.neophotonics.ca · Code: github.com/MatthewLeibel/trueloop-reproduction
June 30, 2026

Abstract

Many control and calibration loops on physical hardware are limited not by computation but by *measurement*: each readout costs real time, photons, shots, dose, or a settling cycle. A gradient-estimating controller must spend several measurements to infer one descent direction — $n{+}1$ for finite differences in n channels, two for SPSA — so when the device drifts and the measurement budget is tight, the controller falls behind a moving target. We describe TrueLoop Compute, a model-free feedback runtime that updates an n-channel control configuration from a *single* measurement per round, and we evaluate it on the task of holding a drifting, nonlinear, multi-channel plant at a setpoint. Driven live against a simulated plant through the production endpoint, at an equal measurement budget the runtime holds a steady-state tracking error of RMS $\approx 2.6 \times 10^{-3}$ that is flat in n from 8 to 256 channels, while finite-difference and SPSA controllers degrade by two orders of magnitude; the advantage grows from $55\times$ to $171\times$ over this range and widens with n. The result rests on no hardware-specific assumption, so it should transfer to a physical device. We provide a self-contained reproduction bundle that runs out of the box on a simulated plant and exposes a single function as the hardware-swap point. We invite experimental groups to reproduce the result on real multi-channel parametric hardware and report what they find, including negative results.

1 Introduction

A gradient-estimating optimizer or controller must spend several objective evaluations to take one informed step. In n dimensions, a forward finite difference costs $n{+}1$ evaluations per step; simultaneous-perturbation stochastic approximation (SPSA) costs two, at the price of much noisier steps. When evaluations are cheap this overhead is invisible. When each evaluation is expensive — a spectrometer integration, a batch of quantum shots, a thermal settling time, a detector pass — the measurement budget becomes the binding constraint, and the loop spends it estimating directions rather than making progress.

This is acute in *regulation under drift*: holding a tunable device at a target while it wanders. Phase stabilization of an interferometric mesh under thermal load, calibration of a multi-channel instrument, holding a sensor array at its operating point — all require correcting faster than the device drifts. A controller that needs $n{+}1$ measurements to compute one correction acquires its gradient over a window in which the target has already moved; at high channel count it never catches up.

We study TrueLoop Compute, a runtime that updates the full n-channel configuration from one measurement per round, model-free and without per-channel tuning. It is intended as the fast inner loop beneath a slower digital strategy. We pose a narrow, falsifiable question: *under a shared measurement budget, against a drifting target, does the runtime hold the setpoint where standard digital controllers cannot?*

2 The measurement-cost argument

The advantage is structural and can be stated before any experiment. Let n be the number of channels and let one *measurement* be a single readout of the plant output vector $\mathbf{y} \in [0,1]^n$ at a configuration $\mathbf{x}$. Consider a fixed budget of B measurements.

A method that produces U control updates from B measurements has a *measurement cost per update*

$$\kappa \;=\; \frac{B}{U} \quad \text{(measurements/update)}. \tag{1}$$

For a controller that estimates a descent direction by perturbing each channel, $\kappa_{\mathrm{FD}} = n + 1$; for SPSA, $\kappa_{\mathrm{SPSA}} = 2$. A runtime that reads the output once and maps it directly to the next configuration has $\kappa_{\mathrm{TL}} = 1$, independent of n. The number of corrective updates available within the budget is therefore

$$U_{\mathrm{FD}} = \frac{B}{n+1}, \qquad U_{\mathrm{TL}} = B. \tag{2}$$

Let the target drift at rate δ, and let each measurement take time t_{m}. A loop holds lock only while it corrects at least as fast as the device drifts. Since each update consumes κ measurements, the achievable correction rate is

$$r \;=\; \frac{1}{\kappa\, t_{\mathrm{m}}}. \tag{3}$$

The maximum trackable drift scales as r, so the trackable-drift ratio between the runtime and a finite-difference controller is

$$\frac{r_{\mathrm{TL}}}{r_{\mathrm{FD}}} = \frac{\kappa_{\mathrm{FD}}}{\kappa_{\mathrm{TL}}} = n + 1. \tag{4}$$

Equation (4) is the entire claim in miniature: at one measurement per update, the runtime corrects up to $n{+}1$ times more often than a finite-difference loop on the same hardware, so its trackable-drift envelope is $(n{+}1)\times$ wider, and the gap *grows with channel count*. The experiments below measure the consequence for steady-state error. No property of the update rule is needed for this argument; only the measurement accounting.

3 Method and protocol

3.1 The runtime, as an interface

The runtime is middleware between the application that sets the objective and the device that realizes it. Each round, the caller supplies the measured output vector $\mathbf{y}_t$ and (for optimization) a scalar score; the runtime returns the next configuration $\mathbf{x}_{t+1}$ to apply. Its defining property is that it never spends a measurement to decide a direction: it reads the current output, compares it to the target, and produces the next configuration directly. The update rule executes

server-side and is never transmitted; only a configuration vector and a scalar cross the wire. Nothing in this brief depends on the rule's internals, and none are disclosed.

The runtime requires only that each channel's response be *locally monotonic* — moving a control moves its observable consistently over the operating range. Sign, gain, and shape are learned from measurement during a brief opening probe; reversed polarity, negative gain, and channels crossing zero need no calibration.

3.2 Task and plant

We evaluate regulation: hold a measured output vector $\mathbf{y} \in [0,1]^n$ at a fixed, reachable setpoint $\mathbf{y}^\star$ while the plant drifts. The plant is multi-channel, nonlinear, and time-varying; per channel

$$y_i(t) = \mathrm{clip}_{[0,1]}\big(\sin[\,x_i\, g_i(t)\,]\big), \qquad (5)$$
$$g_i(t) = g_i^0\,\big[\,1 + \delta \sin(\omega t + \varphi_i)\,\big],$$

with per-channel gains g_i^0 and slow drift of amplitude δ. All physics is simulated; every runtime decision is served live by the production endpoint. The exact plant ships in the bundle and is the object a collaborator replaces with hardware.

3.3 Shared budget and baselines

Every method receives the same total measurement budget B. The runtime runs B rounds at one measurement each. Finite difference consumes B across its $n{+}1$-measurement steps; SPSA across its 2-measurement steps. Baselines are fairly tuned: the finite-difference step and learning rate, and the SPSA gains, were swept and verified to converge given a generous budget, so they are genuinely measurement-starved rather than mis-tuned. The error metric is the steady-state tracking error

$$\mathrm{RMS} = \sqrt{\tfrac{1}{n}\textstyle\sum_{i=1}^{n}\big(y_i - y_i^\star\big)^2}, \qquad (6)$$

evaluated after the budget is exhausted, reported as the median over five independent seeds; "usable" denotes RMS $<$ 0.01.

4 Results

4.1 Equal-budget tracking, scaled in n

Figure 1 reports steady-state tracking RMS at a fixed budget of $B = 80$ measurements as the channel count grows from 8 to 256. The runtime holds RMS $\approx 2.6 \times 10^{-3}$, essentially flat in n, while both gradient baselines degrade by roughly two orders of magnitude: their per-update measurement cost consumes the shared budget before the drifting target settles. The advantage over finite difference (Table 1) rises from 55× at $n = 8$ to 171× at $n = 256$, the widening with n anticipated by Eq. (4).

4.2 Convergence from the first measurements

Figure 2 traces tracking error against cumulative measurements on a single drifting instance ($n = 32$). The runtime crosses into the usable band (RMS $<$ 0.01) within roughly a dozen measurements and holds there; the finite-difference controller, spending $n{+}1$ measurements per step, never reaches the usable band within the same budget. The runtime produces a usable configuration almost immediately and maintains it — the property that matters for an anytime control loop.

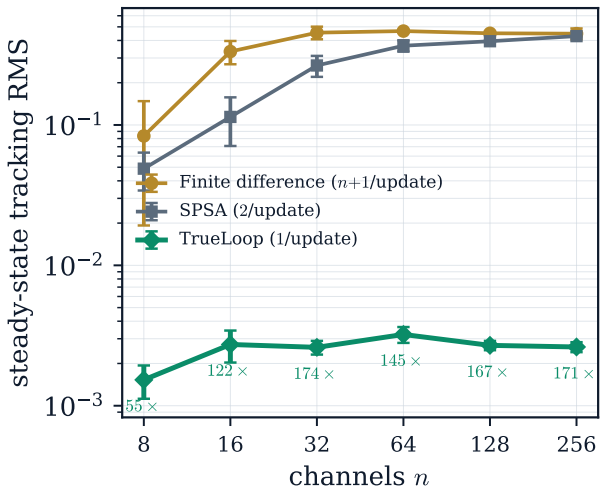

Figure 1. Steady-state tracking RMS at an **equal measurement budget** ($B = 80$), under drift, versus channel count n (log–log). The runtime (teal) is flat in n; finite-difference (gold) and SPSA (grey) degrade as their per-update measurement cost eats the budget. Annotations give the runtime advantage over finite difference. Error bars are ± 1 s.d. over five seeds.

Table 1. Steady-state tracking RMS at equal budget under drift (median of five seeds, live endpoint). "M/upd." is the measurement cost of one corrective update.

n	TrueLoop	Fin. diff.	SPSA	Advantage	M/upd.
8	**0.0015**	0.084	0.049	55×	1 / 9 / 2
16	**0.0027**	0.334	0.114	122×	1 / 17 / 2
32	**0.0026**	0.455	0.265	174×	1 / 33 / 2
64	**0.0032**	0.467	0.367	145×	1 / 65 / 2
128	**0.0027**	0.450	0.395	167×	1 / 129 / 2
256	**0.0026**	0.447	0.429	171×	1 / 257 / 2

4.3 Why the comparison is fair

The runtime extracts a corrective update from each single measurement, so the per-step measurement tax that throttles gradient estimation does not apply to it. The comparison is constructed to favour the baselines: they receive a budget equal to the runtime's; the SPSA baseline is tuned and verified to converge given budget; and no wall-clock element is used — the sole shared constraint is the physical resource of measurements. The runtime wins because it needs fewer of them, which is precisely the property that matters when measurement is the binding constraint. With abundant cheap measurements on a static, well-characterized plant, a well-tuned controller is competitive; this is not a claim of universal superiority.

5 Scope and limitations

We state the boundaries explicitly; they define where the method is expected to succeed.

Where it wins. Drift-robust regulation under a scarce measurement budget; *stiff* plants, where per-channel sensitivities span a wide range and no single fixed gain serves every channel; nonlinear plants, where a fixed-gain controller is mistuned somewhere in the range. These are the regimes the benchmark exercises.

Where it ties. On a linear, stationary, uniform, well-characterized plant with measurements to spare, a well-tuned

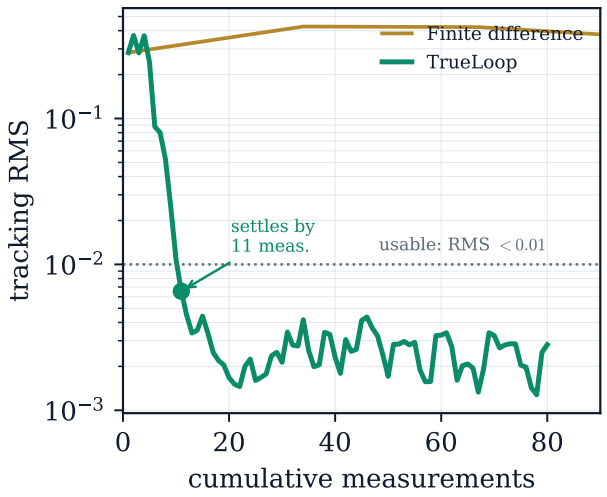

Figure 2. Convergence on one drifting instance ($n = 32$): tracking RMS versus cumulative measurements. The runtime reaches the usable band within ~ 10 measurements and holds it; finite difference does not reach it within the same budget.

PI controller matches it. The runtime's client reports this honestly rather than overstating the fit.

Where it does not apply. Strongly coupled plants, once cross-channel coupling exceeds the diagonally-dominant range (roughly 10–15% of each channel's own response), carry their objective in correlations a per-channel update cannot resolve — a structural limit. Against an instantaneous discontinuity (a sudden full sign reversal of a converged channel), a fixed-sign integrator can recover faster than the runtime's adaptive estimate; its strength is continuous drift and nonlinearity, not abrupt shocks. It has a measurement floor of order ten measurements below which it, too, cannot resolve the setpoint.

What is established, and what is not. All results here are from controlled simulation against fairly-tuned baselines, reproducible on the live endpoint. *Physical hardware validation is the open step*, and the reason for this brief. The measurement-cost argument of §2 rests on no hardware-specific assumption, so we expect the advantage to transfer; confirming that on a real device is the collaboration we propose.

6 Reproduction

The reproduction bundle is self-contained, uses only the Python standard library to run, and reproduces Figures 1–2 on a fresh random instance. It runs out of the box on a simulated plant; one file is the hardware-swap point.

Setup.

1. **Get a key.** Visit compute.neophotonics.ca for a free 90-day evaluation key (this mirror clears common university network filters).
2. **Get the code.** `git clone` the repository at github.com/MatthewLeibel/trueloop-reproduction, then `cd trueloop-reproduction`.
3. **Install the client.** `pip install ./swc_runtime` (a thin HTTP client; no compiled dependencies).
4. **Set your key.** `export TRUELOOP_KEY=EVAL-...`
5. **Run.** `python benchmark.py` — runs on the simulated plant, prints the equal-budget table, writes `results.json`.

Run it on your hardware. Edit the two marked functions in `plant.py`: `apply(x)` writes a control vector to your device, and `measure(x)` reads its output back as a vector in $[0, 1]^n$. Nothing else changes. Algorithm 1 is the entire measurement loop the benchmark runs; your device enters at the single boxed line.

Algorithm 1: The regulation loop (one measurement/round).

Input: key, channels n, setpoint $\mathbf{y}^\star$, budget B
$\mathbf{x} \leftarrow \texttt{start}(\text{key}, n, \mathbf{y}^\star)$
for $t \leftarrow 1$ **to** B **do**
  $\boxed{\mathbf{y} \leftarrow \texttt{measure}(\mathbf{x})}$ `// your device`
  $\mathbf{x} \leftarrow \texttt{step}(\mathbf{y}, \mathbf{y}^\star)$ `// endpoint: next config`
Output: tracking RMS of $\mathbf{y}$ vs. $\mathbf{y}^\star$

What crosses the wire. Only a configuration vector and a scalar per round; raw measurements stay on your machine, and nothing is retained past the session. The runtime's update rule runs server-side and is never downloaded. The repository contains the open reproduction harness only; the runtime method is proprietary and is not included in or licensed by it.

7 What collaboration looks like

We are not seeking funding, exclusivity, or a long commitment — only an honest run on a real multi-channel parametric device (a photonic mesh, an RF front-end or phased array, a tunable optical or microwave instrument, a multi-channel calibration target) and a short account of what you observe. The scope is deliberately small and time-boxed: one question, an agreed pass/fail bar, one clear result. We will support the integration directly, share the full methodology behind any number here, and credit your group as you prefer. A negative result — the advantage failing to transfer to your hardware — is a valid, reportable outcome, and one we want to hear.

Contact. matthew@trueloopcompute.com · Key: compute.neophotonics.ca · Code: github.com/MatthewLeibel/trueloop-reproduction · NEOTECH Inc., Port Coquitlam, BC.